

Отчёт по лабораторной работе №9

Понятие подпрограммы. Отладчик GDB.

Мещерякова София Андреевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
2.1	Реализация подпрограмм в NASM	5
2.2	Отладка программ с помощью GDB	8
3	Выводы	24

Список иллюстраций

2.1	Создаем каталог и файл для лабораторной работы №9	5
2.2	Текст программы (Листинг 9.1)	6
2.3	Запуск программы	6
2.4	Текст программы	7
2.5	Запуск программы	7
2.6	Запуск программы	8
2.7	Текст программы (Листинг 9.2)	8
2.8	Получаем исполняемый файл	9
2.9	Запуск файла в отладчике (команда run)	9
2.10	Установка брейкпоинта	9
2.11	Дисассимилированный код программы	10
2.12	Синтаксис Intel	11
2.13	Отображение регистров и их значений, результат дисассимилирования программы	13
2.14	info breakpoints - для просмотра информации о о точках останова . .	13
2.15	Новый брейкпоинт	14
2.16	Информация о брейкпоинтах	14
2.17	Отслеживаем значения регистров	15
2.18	Значение переменной msg1	15
2.19	Значение переменной msg2	16
2.20	Изменение значения переменной msg1	16
2.21	Изменение значения переменной msg2	16
2.22	Значение регистра edx	16
2.23	Изменения регистра ebx	16
2.24	Выход из gdb (команды с и q)	17
2.25	Копирование файла	17
2.26	Создаем исполняемый файл	17
2.27	Згружаем файл в отладчик	17
2.28	Создание брейкпоинта и запуск программы	17
2.29	Создание брейкпоинта и запуск программы	18
2.30	Значения позиций стека	18
2.31	Текст программы	19
2.32	Запуск программы	19
2.33	Текст программы	20
2.34	Запуск программы	21
2.35	Ищем ошибку с помощью отладчика	21

2.36	Текст программы с исправлениями	22
2.37	Запуск программы	22

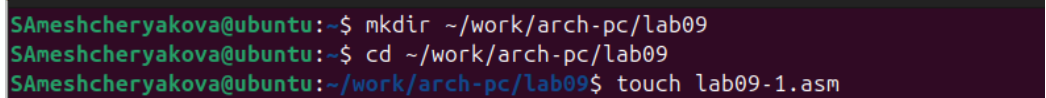
1 Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Выполнение лабораторной работы

2.1 Реализация подпрограмм в NASM

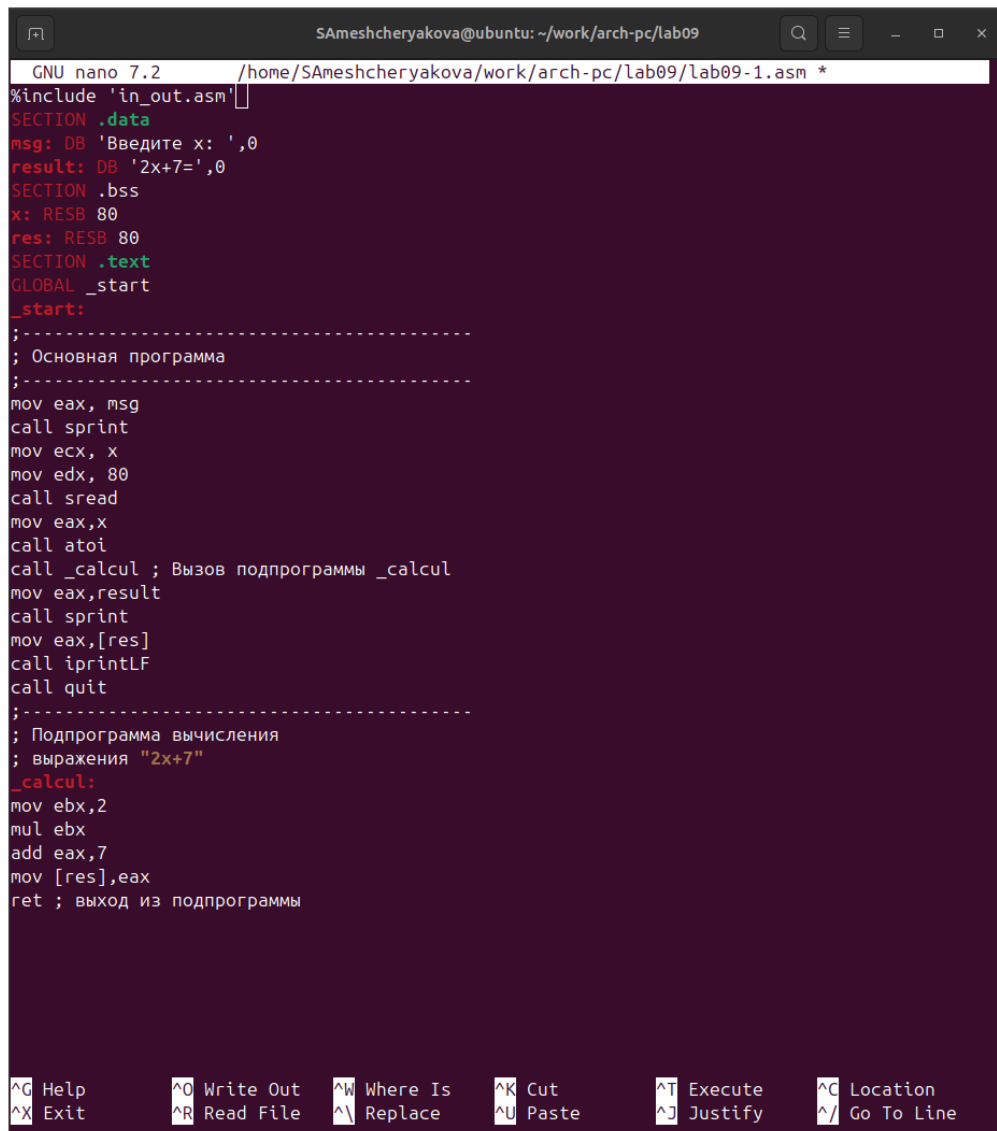
1. Создадим каталог для программ лабораторной работы № 9, перейдём в него и создадим файл lab09-1.asm (рис. 2.1).



```
SAmeshcheryakova@ubuntu:~$ mkdir ~/work/arch-pc/lab09
SAmeshcheryakova@ubuntu:~$ cd ~/work/arch-pc/lab09
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ touch lab09-1.asm
```

Рисунок 2.1: Создаем каталог и файл для лабораторной работы №9

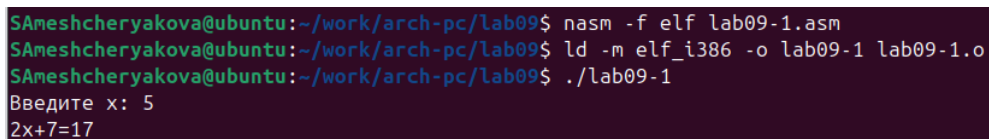
Открываем файл и вводим программу из листинга 9.1 (рис. 2.2).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-1.asm *
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit
;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
mov ebx, 2
mul ebx
add eax, 7
mov [res], eax
ret ; выход из подпрограммы
```

Рисунок 2.2: Текст программы (Листинг 9.1)

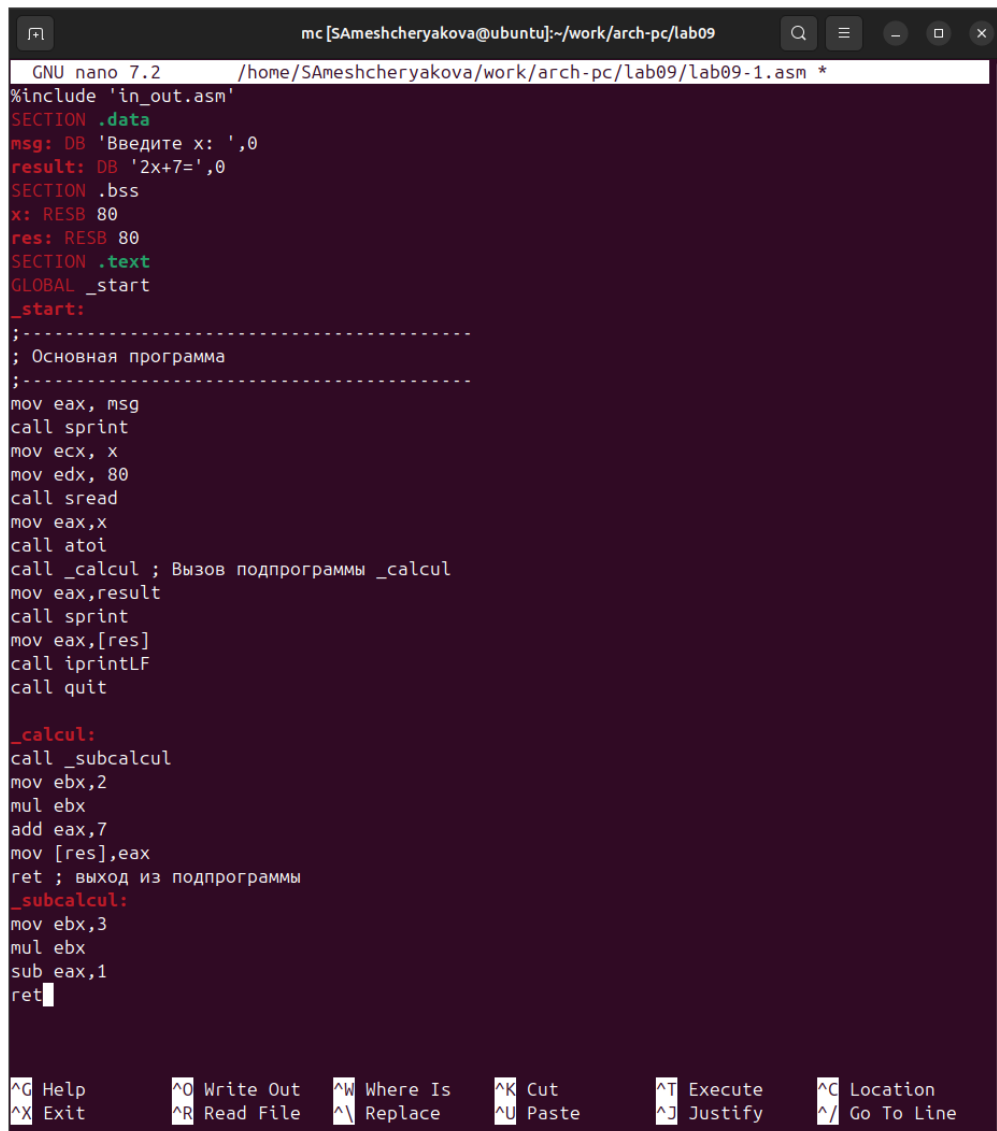
Создаем исполняемый файл и запускаем его (рис. 2.3).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x+7=17
```

Рисунок 2.3: Запуск программы

Изменяем файл, добавив подпрограмму _subcalcul (рис. 2.4).

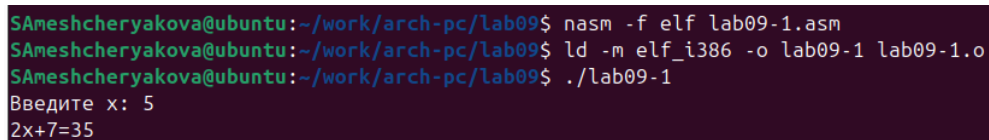


```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab09
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-1.asm *
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit

_calcul:
call _subcalcul
mov ebx, 2
mul ebx
add eax, 7
mov [res], eax
ret ; выход из подпрограммы
_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret
```

Рисунок 2.4: Текст программы

Создаем исполняемый файл и запускаем его (рис. 2.5).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x+7=35
```

Рисунок 2.5: Запуск программы

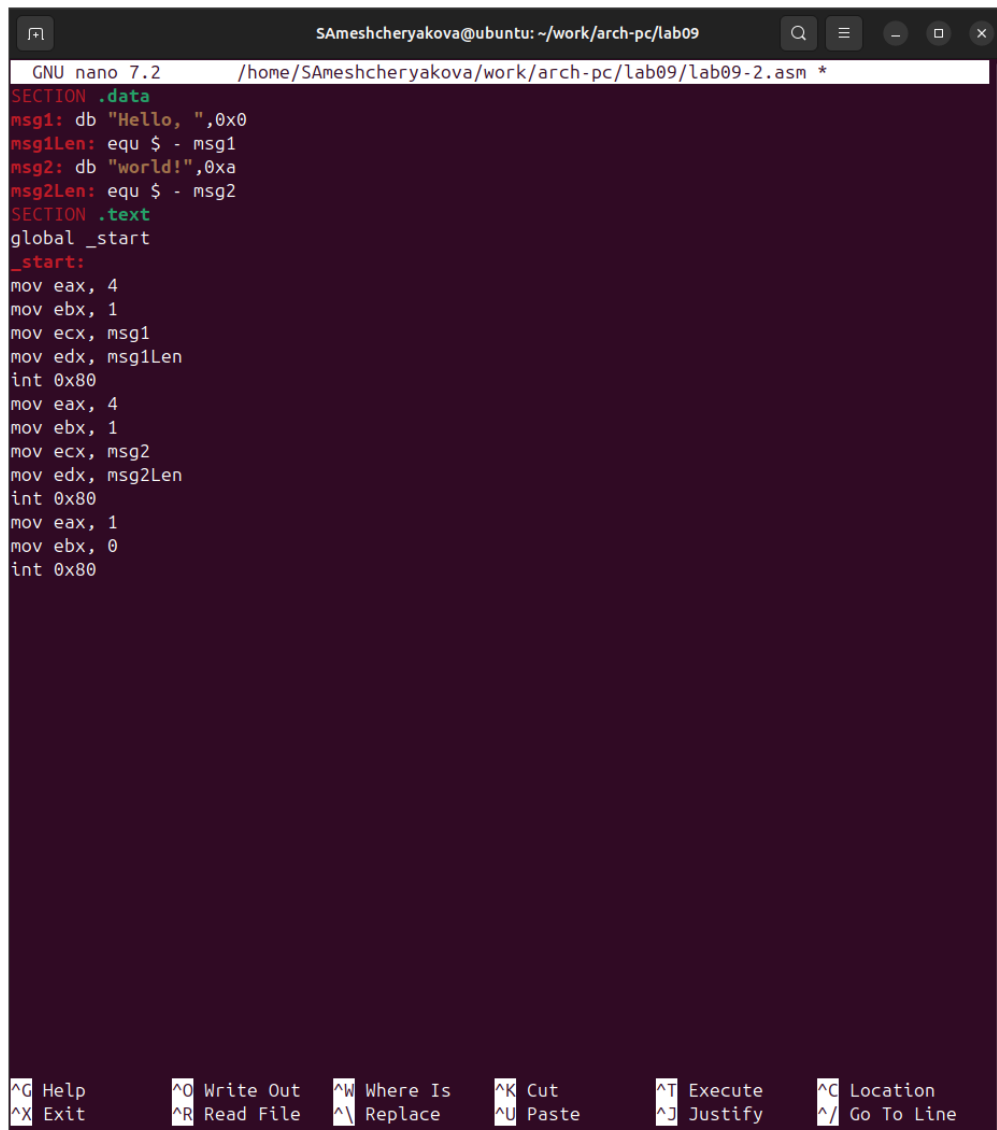
2.2 Отладка программ с помощью GDB

Создадим файл lab09-2.asm (рис. 2.6).

```
SAmeshcheryakova@ubuntu: ~/work/arch-pc/lab09$ touch lab09-2.asm
```

Рисунок 2.6: Запуск программы

Открываем файл и вводим программу из листинга 9.2 (рис. 2.7).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-2.asm *
SECTION .data
msg1: db "Hello, ",0x0
msg1len: equ $ - msg1
msg2: db "world!",0xa
msg2len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Рисунок 2.7: Текст программы (Листинг 9.2)

Получим исходный файл с использованием отладчика gdb (рис. 2.8).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
```

Рисунок 2.8: Получаем исполняемый файл

Загрузим файл в отладчик и запустим в нем программу (?@fig-009).

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb)
```

Рисунок 2.9: Запуск файла в отладчике (команда run)

Устанавливаем брейкпоинт на метку _start и запускаем программу (рис. 2.10).

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb)
```

Рисунок 2.10: Установка брейкпоинта

Смотрим дисассимилированный код программы, начиная с метки _start (рис. 2.11).

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
    0x08049005 <+5>:      mov     $0x1,%ebx
    0x0804900a <+10>:     mov     $0x804a000,%ecx
    0x0804900f <+15>:     mov     $0x8,%edx
    0x08049014 <+20>:     int     $0x80
    0x08049016 <+22>:     mov     $0x4,%eax
    0x0804901b <+27>:     mov     $0x1,%ebx
    0x08049020 <+32>:     mov     $0x804a008,%ecx
    0x08049025 <+37>:     mov     $0x7,%edx
    0x0804902a <+42>:     int     $0x80
    0x0804902c <+44>:     mov     $0x1,%eax
    0x08049031 <+49>:     mov     $0x0,%ebx
    0x08049036 <+54>:     int     $0x80
End of assembler dump.
```

Рисунок 2.11: Дисассимилированный код программы

Переключаемся на отображение команд с Intel'овским синтаксисом (рис. 2.12).

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a008
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.

```

Рисунок 2.12: Синтаксис Intel

Различия отображения синтаксиса машинных команд в режимах ATT и Intel:

1. Порядок операндов:

ATT: исходный операнд, затем результирующий, Intel: результирующий операнд, затем исходный.

2. Разделители:

ATT: запятые, Intel: запятые или косые черты (/).

3. Указание размера операндов:

ATT: префиксы перед операндом (b, w, l, q), Intel: суффиксы после операнда (b, w, d, q).

4. Знак операндов:

АТТ: положительные значения начинаются с \$, Intel: положительные значения без \$.

5. Обозначение адресов:

АТТ: в круглых скобках, Intel: без скобок.

6. Обозначение регистров:

АТТ: с символом % (например, %eax), Intel: без %, может начинаться с R или E (например, RAX, EAX).

Включаем режим псевдографики (рис. 2.13).

```
SAMeshcheryakova@ubuntu: ~/work/arch-pc/lab09
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf70 0xffffcf70
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049000 0x8049000 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43

B> 0x8049000 <_start> mov eax,0x4
0x8049005 <_start+5> mov ebx,0x1
0x804900a <_start+10> mov ecx,0x804a000
0x804900f <_start+15> mov edx,0x8
0x8049014 <_start+20> int 0x80
0x8049016 <_start+22> mov eax,0x4
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x7
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1
0x8049031 <_start+49> mov ebx,0x0
0x8049036 <_start+54> int 0x80

native process 8951 (asm) In: _start L9 PC: 0x8049000
(gdb) layout regs
(gdb) █
```

Рисунок 2.13: Отображение регистров и их значений, результат дисассимилирования программы

Проверяем была ли установлена точка остановки (рис. 2.14).

```
(gdb) info breakpoints
Num      Type           Disp Enb Address      What
1        breakpoint      keep y  0x08049000 lab09-2.asm:9
breakpoint already hit 1 time
```

Рисунок 2.14: info breakpoints - для просмотра информации о о точках останова

Установим еще один брейкпоинт (рис. 2.15).

```
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 20.
```

Рисунок 2.15: Новый брейкпоинт

Посмотрим информацию о всех установленных точках останова (рис. 2.16).

```
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x08049000 lab09-2.asm:9
          breakpoint already hit 1 time
2        breakpoint     keep y   0x08049031 lab09-2.asm:20
(gdb)
```

Рисунок 2.16: Информация о брейкпоинтах

Выполняем 5 инструкций командой si (рис. 2.17).

```

Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffcf70 0xffffcf70
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43

B+ 0x8049000 <_start>    mov     eax,0x4
0x8049005 <_start+5>    mov     ebx,0x1
0x804900a <_start+10>   mov     ecx,0x804a000
0x804900f <_start+15>   mov     edx,0x8
0x8049014 <_start+20>   int     0x80
>0x8049016 <_start+22>  mov     eax,0x4
0x804901b <_start+27>  mov     ebx,0x1
0x8049020 <_start+32>  mov     ecx,0x804a008
0x8049025 <_start+37>  mov     edx,0x7
0x804902a <_start+42>  int     0x80
0x804902c <_start+44>  mov     eax,0x1
b+ 0x8049031 <_start+49> mov     ebx,0x0
0x8049036 <_start+54>  int     0x80

native process 8951 (asm) In: _start L14 PC: 0x8049016
1 breakpoint keep y 0x08049000 lab09-2.asm:9
  breakpoint already hit 1 time
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 20.
(gdb) i b
Num   Type           Disp Enb Address      What
1     breakpoint      keep y  0x08049000 lab09-2.asm:9
      breakpoint already hit 1 time
2     breakpoint      keep y  0x08049031 lab09-2.asm:20
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb)

```

Рисунок 2.17: Отслеживаем значения регистров

Во время выполнения команд менялись регистры: ebx, ecx, edx, eax, eip.

Смотрим значение переменной msg1 по имени (рис. 2.18).

```

(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "

```

Рисунок 2.18: Значение переменной msg1

Смотрим значение переменной msg2 по адресу (рис. 2.19).


```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>: "world!\n\034"
```

Рисунок 2.19: Значение переменной msg2

Изменим первый символ переменной msg1 (рис. 2.20).

```
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>: "hello, "
```

Рисунок 2.20: Изменение значения переменной msg1

Изменим первый символ переменной msg2 (рис. 2.21).

```
(gdb) set {char}&msg2='W'
(gdb) x/1sb &msg2
0x804a008 <msg2>: "World!\n\034"
```

Рисунок 2.21: Изменение значения переменной msg2

Смотрим значение регистра edx в разных форматах (рис. 2.22).

```
(gdb) p/t $edx
$1 = 1000
(gdb) p/s $edx
$2 = 8
(gdb) p/x $edx
$3 = 0x8
```

Рисунок 2.22: Значение регистра edx

Изменяем регистр ebx (рис. 2.23).

```
(gdb) set $ebx='2'
(gdb) p/s $ebx
$4 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$5 = 2
```

Рисунок 2.23: Изменения регистра ebx

Выводятся разные значения, так как команда без кавычек присваивает регистру числовое значение (его и выводит), а „2“ - строковое значение.

Прописываем команды для завершения программы и выхода из GDB (рис. 2.24).

```
(gdb) c
Continuing.
World!

Breakpoint 2, _start () at lab09-2.asm:20
(gdb) q
```

Рисунок 2.24: Выход из gdb (команды c и q)

Копируем файл lab8-2.asm в файл с именем lab09-3.asm (рис. 2.25).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
```

Рисунок 2.25: Копирование файла

Создадим исполняемый файл и загрузим его в отладчик GDB (рис. 2.26 , 2.27).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
```

Рисунок 2.26: Создаем исполняемый файл

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ gdb --args lab09-3 2 3 '5'
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
```

Рисунок 2.27: Згружаем файл в отладчик

Установим точку останова перед первой инструкцией в программе и запустим ее (рис. 2.28).

```
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 5.
(gdb) run
Starting program: /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-3 2 3 5

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000
Download failed: No route to host. Continuing without separate debug info for system-supplied DSO at 0xf7ffc000.

Breakpoint 1, _start () at lab09-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
```

Рисунок 2.28: Создание брейкпоинта и запуск программы

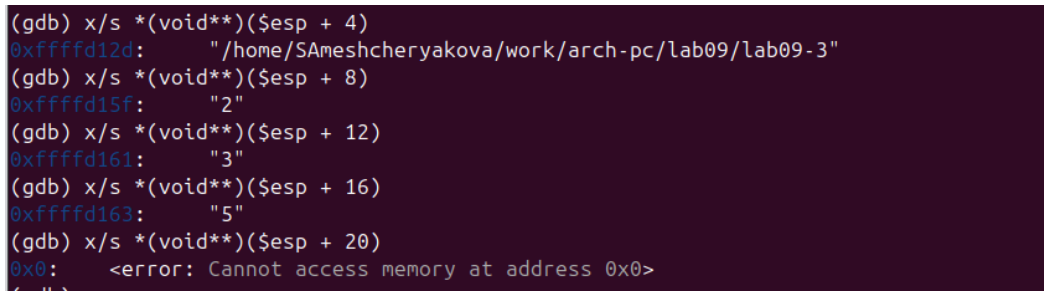
Смотрим адрес вершины стека (рис. 2.29).



```
(gdb) x/x $esp
0xffffcf50: 0x00000004
```

Рисунок 2.29: Создание брейкпоинта и запуск программы

Смотрим значения стека по разным адресам (рис. 2.30).



```
(gdb) x/s *(void**)($esp + 4)
0xffffd12d: "/home/SAmeshcheryakova/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)($esp + 8)
0xffffd15f: "2"
(gdb) x/s *(void**)($esp + 12)
0xffffd161: "3"
(gdb) x/s *(void**)($esp + 16)
0xffffd163: "5"
(gdb) x/s *(void**)($esp + 20)
0x0: <error: Cannot access memory at address 0x0>
```

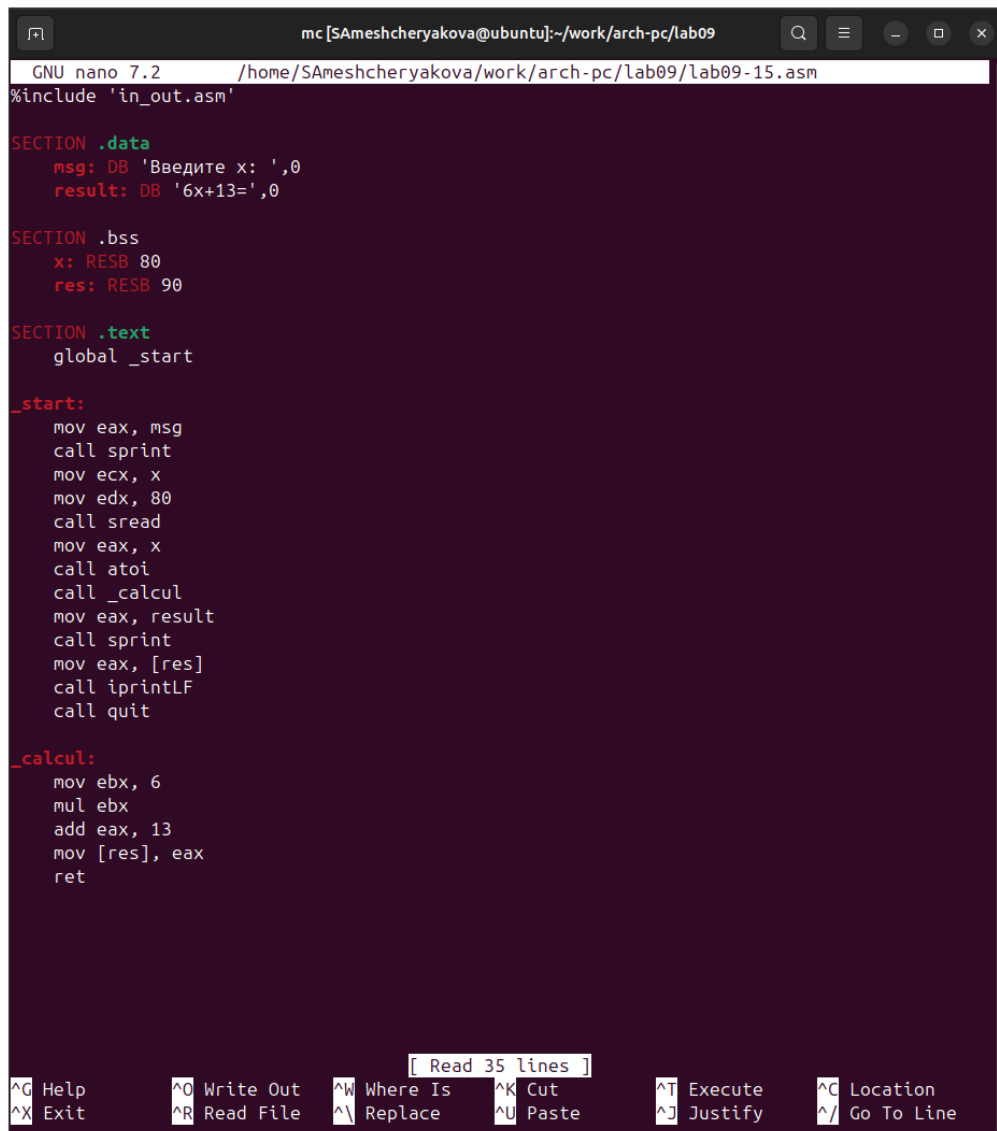
Рисунок 2.30: Значения позиций стека

Шаг изменения адреса равен 4 потому что адресные регистры имеют размерность 4 байта.

##Задание для самостоятельной работы

1. Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму.

Открываем файл lab09-15.asm (копия файла lab8-15.asm) и изменим его (рис. 2.31).



```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab09
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-15.asm
#include 'in_out.asm'

SECTION .data
msg: DB 'Введите x: ',0
result: DB '6x+13=',0

SECTION .bss
x: RESB 80
res: RESB 90

SECTION .text
global _start

_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    call _calcul
    mov eax, result
    call sprint
    mov eax, [res]
    call iprintLF
    call quit

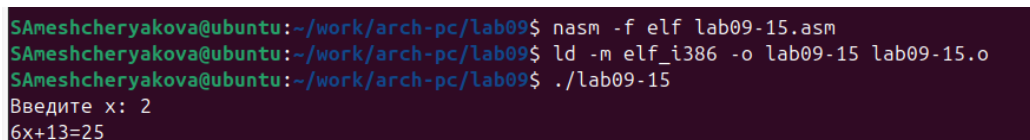
_calcul:
    mov ebx, 6
    mul ebx
    add eax, 13
    mov [res], eax
    ret
```

[Read 35 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^ Replace	^U Paste	^J Justify	^_ Go To Line

Рисунок 2.31: Текст программы

Создадим исполняемый файл и запустим его (рис. 2.32).



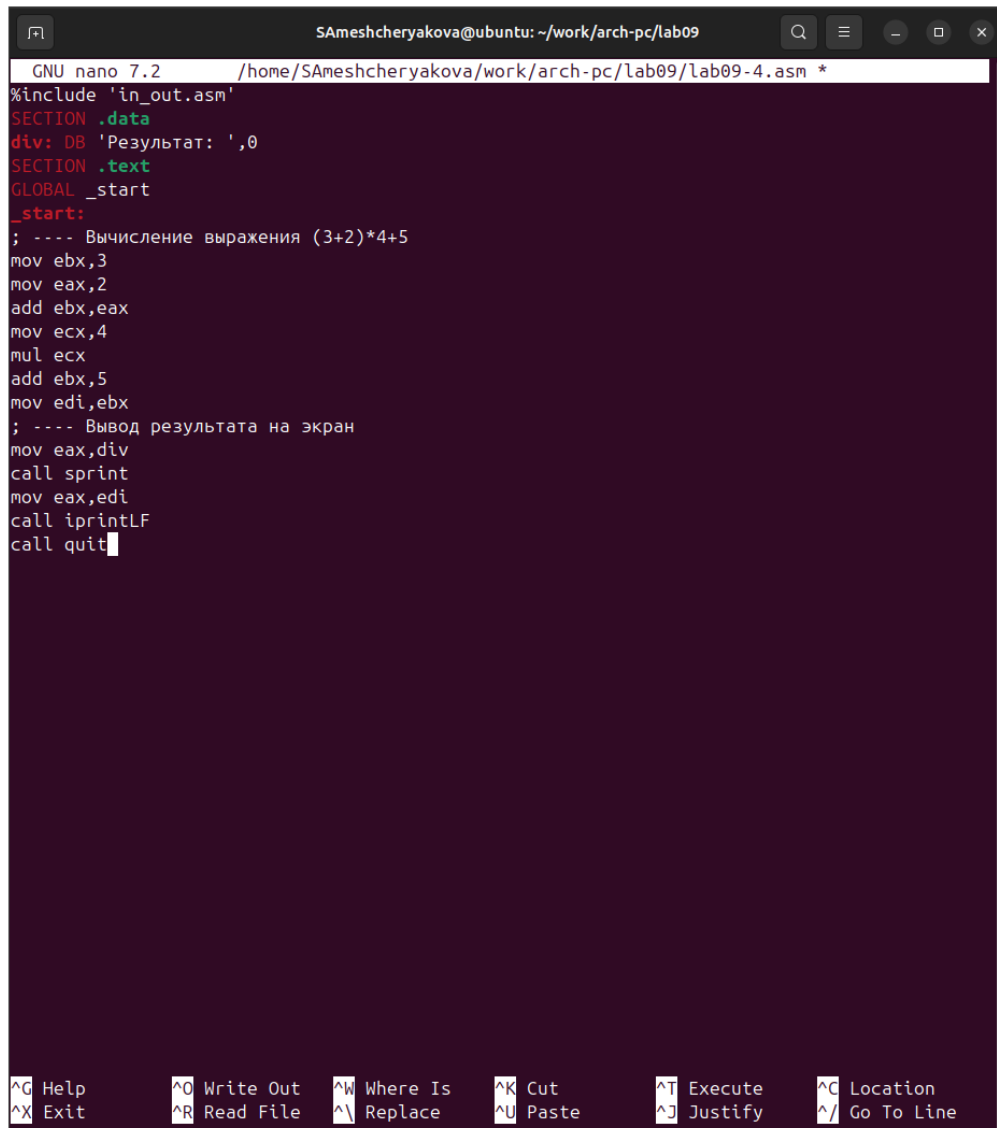
```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf lab09-15.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-15 lab09-15.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ./lab09-15
Введите x: 2
6x+13=25
```

Рисунок 2.32: Запуск программы

- В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$.

При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Создаем новый файл и вводим туда текст программы из листинга 9.3 (рис. 2.33).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-4.asm *
#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Terminal window showing the assembly code for a program in the nano editor. The code includes a data section with a string 'Результат: ', a text section with the start label, and assembly instructions to calculate (3+2)*4+5 and print the result. The bottom of the window shows nano editor shortcuts.

Рисунок 2.33: Текст программы

Создадим исполняемый файл и запустим его (рис. 2.34).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf lab09-4.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-4 lab09-4.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ./lab09-4
Результат: 10

```

Рисунок 2.34: Запуск программы

Создаем исполняемый файл и запускаем его в отладчике GDB и смотрим на изменение регистров командой `si` (рис. 2.35).

The screenshot shows a GDB terminal window with the following content:

```

SAmeshcheryakova@ubuntu: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8
ecx      0x4      4
edx      0x0      0
ebx      0x5      5
esp      0xffffcf60 0xffffcf60
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x80490fb 0x80490fb <_start+19>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43

B+ 0x80490e8 <_start>    mov     $0x3,%ebx
0x80490ed <_start+5>    mov     $0x2,%eax
0x80490f2 <_start+10>   add     %eax,%ebx
0x80490f4 <_start+12>   mov     $0x4,%ecx
0x80490f9 <_start+17>   mul     %ecx
>0x80490fb <_start+19>  add     $0x5,%ebx
0x80490fe <_start+22>   mov     %ebx,%edi
0x8049100 <_start+24>   mov     $0x804a000,%eax
0x8049105 <_start+29>   call   0x804900f <sprint>
0x804910a <_start+34>   mov     %edi,%eax
0x804910c <_start+36>   call   0x8049086 <iprintLF>
0x8049111 <_start+41>   call   0x80490db <quit>
0x8049116             add     %al,(%eax)

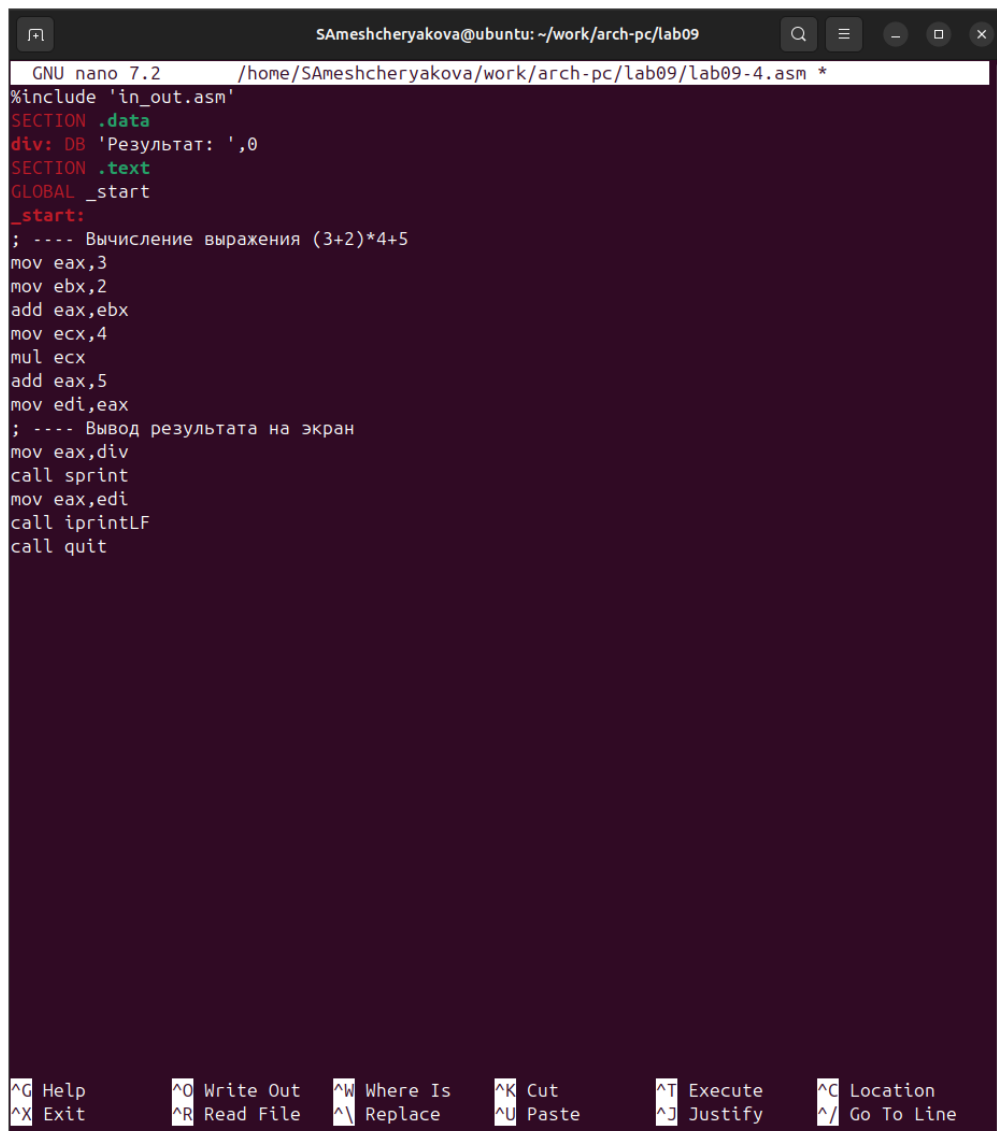
native process 11466 (asm) In: _start L13 PC: 0x80490fb
(gdb) layout regs
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-4.asm, line 8.
(gdb) run
Starting program: /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-4

Breakpoint 1, _start () at lab09-4.asm:8
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb)

```

Рисунок 2.35: Ищем ошибку с помощью отладчика

Изменяем программу для корректной работы (рис. 2.36).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab09/lab09-4.asm *
#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov eax,3
mov ebx,2
add eax,ebx
mov ecx,4
mul ecx
add eax,5
mov edi,eax
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Рисунок 2.36: Текст программы с исправлениями

Создадим исполняемый файл и запустим его (рис. 2.37).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ nasm -f elf lab09-4.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-4 lab09-4.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab09$ ./lab09-4
Результат: 25
```

Рисунок 2.37: Запуск программы

Теперь программа работает корректно.

3 Выводы

Мы познакомились с методами отладки при помощи GDB и его возможностями, а также научились работать с подпрограммами.